

1.1.2.1 Monitoring et supervision

EN BREF

Le **monitoring** sur Hublot permet de **détecter les problèmes avant les utilisateurs** : site indisponible, build cassé, erreurs JavaScript en production. Nous combinons une **sonde de disponibilité** (`/health.json`), des **alertes CI/CD** (GitHub, Netlify), des **tests E2E** en filet de sécurité, et **Sentry** (optionnel) pour les erreurs applicatives. Ce chapitre explique chaque brique, pourquoi elle existe, et comment l'activer.

Objectif pédagogique

Comprendre les bases du **monitoring et de la supervision** dans une démarche DevOps : disponibilité, erreurs, signaux CI/CD, et outils de sonde.

Contexte — projet Hublot

Hublot est consulté par des agents DIRM pour le **pilotage RH**. Si l'application est down ou affiche des données incohérentes sans alerte, les utilisateurs découvrent le problème par eux-mêmes — trop tard pour une application institutionnelle. Comme Hublot est un **front statique** (pas de serveur Node en prod), le monitoring se concentre sur :

- **Disponibilité** — le site répond-il ?
- **Qualité avant deploy** — la CI est-elle verte ?
- **Erreurs côté navigateur** — Sentry remonte-t-il des exceptions ?

Définitions

Terme	Définition
Monitoring	Surveillance continue de l'état d'un service (disponible, lent, en erreur).
Supervision	Processus humain ou outillé qui alerte et agit quand un seuil est dépassé.
Sonde (health check)	Requête périodique vers un endpoint (ici <code>/health.json</code>) pour vérifier que l'app vit.
Observabilité	Capacité à comprendre l'état interne du système via logs, métriques, traces (Sentry = partiel pour le front).
Uptime	Pourcentage de temps où le service est accessible.
DSN (Sentry)	URL de projet permettant au client JavaScript d'envoyer les erreurs à Sentry.

Ce qui a été mis en place

1. Sonde de disponibilité — `public/health.json`

Un fichier JSON statique est servi à la racine du site. Il ne contient **aucune donnée RH** ; il indique seulement que le front est en ligne.

URL production : `https://dirmhublot.netlify.app/health.json`

Contenu attendu :

JSON

```
{
  "status": "ok",
  "app": "hublot",
  "service": "statdirm-frontend"
}
```

Pourquoi un fichier statique ? Netlify sert les fichiers du dossier `public/` sans logique serveur. C'est simple, fiable, et compatible avec des outils externes (UptimeRobot, curl, scripts de supervision).

2. Erreurs applicatives — Sentry (optionnel)

Le module `src/monitoring/sentry.ts` est chargé depuis `main.tsx` **uniquement** si la variable `VITE_SENTRY_DSN` est définie. Sans DSN, **aucun appel réseau** Sentry n'est effectué (zéro impact performance).

Activation :

1. Créer un projet React sur sentry.io (<https://sentry.io>)
2. Netlify → Site settings → Environment variables :

- `VITE_SENTRY_DSN` = DSN du projet
- `VITE_APP_ENV` = `production` | `staging` | `preview`

1. Redéployer le site

3. Signaux CI / CD

Signal	Où le voir	Signification
Échec lint / tests / E2E	GitHub Actions → workflow CI	Code ou régression détectée avant prod
Échec audit npm	Job Audit npm	Dépendance vulnérable
Échec deploy	Netlify → Deploys → logs	Build ou config Netlify incorrecte
NAS hors ligne	Sonde sur <code>http://IP_NAS:8080/health.json</code>	Hébergement interne indisponible

GitHub envoie un **email** en cas d'échec de workflow si les notifications sont activées.

4. Tests E2E comme filet

Les scénarios Playwright (`e2e/smoke.spec.ts`) vérifient en CI :

- Affichage de la page de connexion
- Parcours login → tableau de bord
- Réponse de `/health.json`

Ils complètent le monitoring externe : même si `/health.json` répond, un bug d'authentification serait détecté par l'E2E avant merge.

Pourquoi ces choix

Choix	Justification
<code>/health.json</code> plutôt qu'une API lourde	Front statique : pas de backend dédié en prod Netlify.
Sentry optionnel	Environnements de dev/test n'envoient pas de bruit ; la prod peut activer le DSN.
CI comme premier monitoring	Mieux vaut empêcher un bug d'arriver en prod que le détecter après.
Pas de Datadog/Grafana	Hors scope et coût pour un SPA statique ; évolution possible si backend dédié.

Comment ça fonctionne

Configurer une alerte UptimeRobot (exemple)

- 1 Créer un compte sur uptimerobot.com (<https://uptimerobot.com>)
- 2 Nouveau monitor **HTTP(s)**
- 3 URL : `https://dirmhublot.netlify.app/health.json`
- 4 Intervalle : 5 minutes
- 5 Mot-clé dans la réponse : `"status":"ok"`
- 6 Alerte : email ou Slack

Vérifier manuellement

TERMINAL

```
curl -s https://dirmhublot.netlify.app/health.json  
npm run test:e2e # inclut la vérif health en CI locale
```

Lire une alerte Sentry (si activé)

- 1 Connexion sentry.io → projet Hublot
- 2 Issues → stack trace, navigateur, URL, fréquence
- 3 Corréler avec le deploy Netlify de la même période

Quand

Fréquence	Action
Continu (externe)	UptimeRobot ping <code>/health.json</code>
À chaque push	CI + E2E (dont health)
À chaque erreur JS (si Sentry actif)	Remontée vers Sentry
Après chaque deploy Netlify	Vérifier Deploy log + curl health

Ce qui n'est pas en place (limites assumées)

- **Datadog / Grafana** : non intégrés — pertinents surtout avec serveur et métriques infra.
- **Logs centralisés serveur** : front statique ; pas de flux applicatif backend hors Functions Netlify éventuelles.

SYNTHÈSE

« Le monitoring de Hublot repose sur la **sonde** `/health.json`, les **alertes CI/CD** (GitHub, Netlify) et **Sentry** en option pour les erreurs navigateur. L'objectif n'est pas d'attendre qu'un agent signale un dysfonctionnement, mais d'être **alerté dès qu'un contrôle échoue** — disponibilité, build ou parcours critique login. »

Documents liés

- › 1.1.2 La démarche DevOps — chaîne CI/CD
- › 1.2.10 Guide Playwright — tests E2E
- › 1.2.8 Procédure d'exécution des tests (épreuve).md) — démo jury